

QECTOR Decoder v3

Master Benchmark Report — v0.6.7

CPU + CUDA GPU · code-capacity throughput & circuit-level LER

Author / owner : Guillaume Lessard (admin@qector.store)

Generated (UTC): 2026-07-22T18:50:46.516309+00:00

Host : LAPTOP-BCRBUPRO

OS : Windows-10-10.0.26100-SP0

CPU : AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD (16 logical)

RAM : 16.5 GB

GPU : NVIDIA GeForce GTX 1660 Ti, 610.62, 6144 MiB, 4336 MiB, 1631 MiB

Python : 3.11.0

Wheel : qector_decoder_v3.cp311-win_amd64.pyd

Wheel sha256 : e3a24f442cc595dc73e0821ff32e3a09...

numpy 2.2.6 · stim 1.16.0 · pymatching 2.4.0

Noise models:

- Throughput matrix — code-capacity i.i.d. bit-flip on the single-sector rotated surface code (decoder micro-benchmark; scales to 200k shots).
- Accuracy / LER — circuit-level depolarizing noise via Stim surface_code:rotated_memory_z, decomposed DEM, documented dem.from_stim API.

Executive Summary

KEY FINDINGS

1. CRITICAL PACKAGING BUG (v0.6.7): ``import qector_decoder_v3`` raises `AttributeError: module '...qector_decoder_v3' has no attribute 'HybridCascadeDecoder'` (`__init__.py` line 37 is not guarded by `try/except`, unlike every other optional native symbol). The shipped Windows cp311 wheel's compiled core does NOT export that symbol, so a clean install cannot import the package at all. Every documented submodule (`dem`, `codes`, `stim_compat`, `sinter_compat`, `bposd`) is unreachable.
-> Fix: guard line 37 exactly like `LERBenchmark/CUDA/OpenCL` above it.
2. This report was produced by loading the compiled core directly and, for the documented API, injecting the missing symbol in-memory. Installed files were left pristine (bug remains reproducible).
3. Throughput: peak CPU $\sim 5,212,664$ shots/s, peak CUDA $\sim 13,487,996$ shots/s on a GTX 1660 Ti (6 GB, ~ 1.6 GB free under background load). CUDA dominates at small-to-mid distance; but the CPU Union-Find OVERTAKES CUDA at large distance+batch on this entry GPU (`d11@200k GPU  $\sim 0.62\times$  CPU, d19@200k  $\sim 0.46\times$`).
4. Faithfulness: EVERY decoder (incl. CUDA) reproduces the syndrome ($H \cdot c = s$) on 100% of shots at all distances `d3..d19`.
5. 'CUDA bit-identical to CPU' — VERIFIED: CUDA corrections are 100% row-exact vs the CPU Union-Find at every distance `d3..d19` (20k shots each). CUDA vs Blossom diverges with distance (UF != weighted MWPM; both stay faithful).
6. Circuit-level LER (Stim rotated_memory_z, decomposed DEM): QECTOR's weighted MWPM matches PyMatching failure-for-failure and shows genuine error suppression with distance; unweighted Union-Find (and thus CUDA) does not suppress at circuit level (expected – it ignores edge weights).
7. Other stub/binary mismatches: `OpenCLBatchDecoder / opencl_is_available` absent from this wheel; native `BlossomDecoder` requires `edge_weights` positionally (public wrapper supplies it); `CPUBatchDecoder` exposes only `batch_decode_par` (no `batch_decode`).

Environment & Reproducibility

ENVIRONMENT & REPRODUCIBILITY

Host : LAPTOP-BCRBUPRO
OS : Windows-10-10.0.26100-SP0
CPU : AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD (16 logical cores)
RAM : 16.5 GB
GPU : NVIDIA GeForce GTX 1660 Ti, 610.62, 6144 MiB, 4336 MiB, 1631 MiB
CUDA avail : True
Python : 3.11.0
Wheel file : qector_decoder_v3.cp311-win_amd64.pyd
Wheel sha256: e3a24f442cc595dc73e0821ff32e3a09d7cd518b6bb5786966e5fb7760d16676

Package versions:

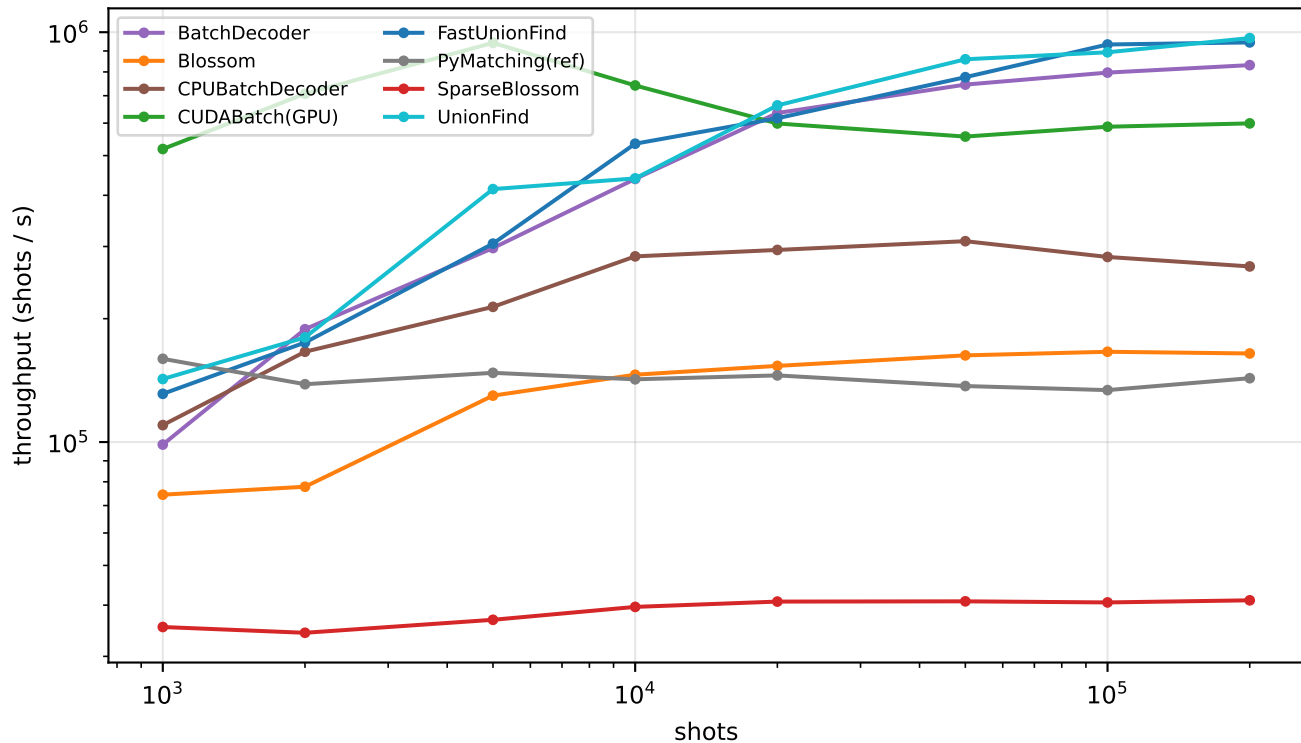
qector-decoder-v3	0.6.7
numpy	2.2.6
pymatching	2.4.0
stim	1.16.0
sinter	1.16.0
ldpc	2.4.1
beliefmatching	0.2.0
scipy	1.17.1
cryptography	49.0.0

Seed : 20260722
Distances : [3, 5, 7, 9, 11, 13, 15, 17, 19]
Shot counts : [1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000]
p_throughput: 0.05 (code-capacity)

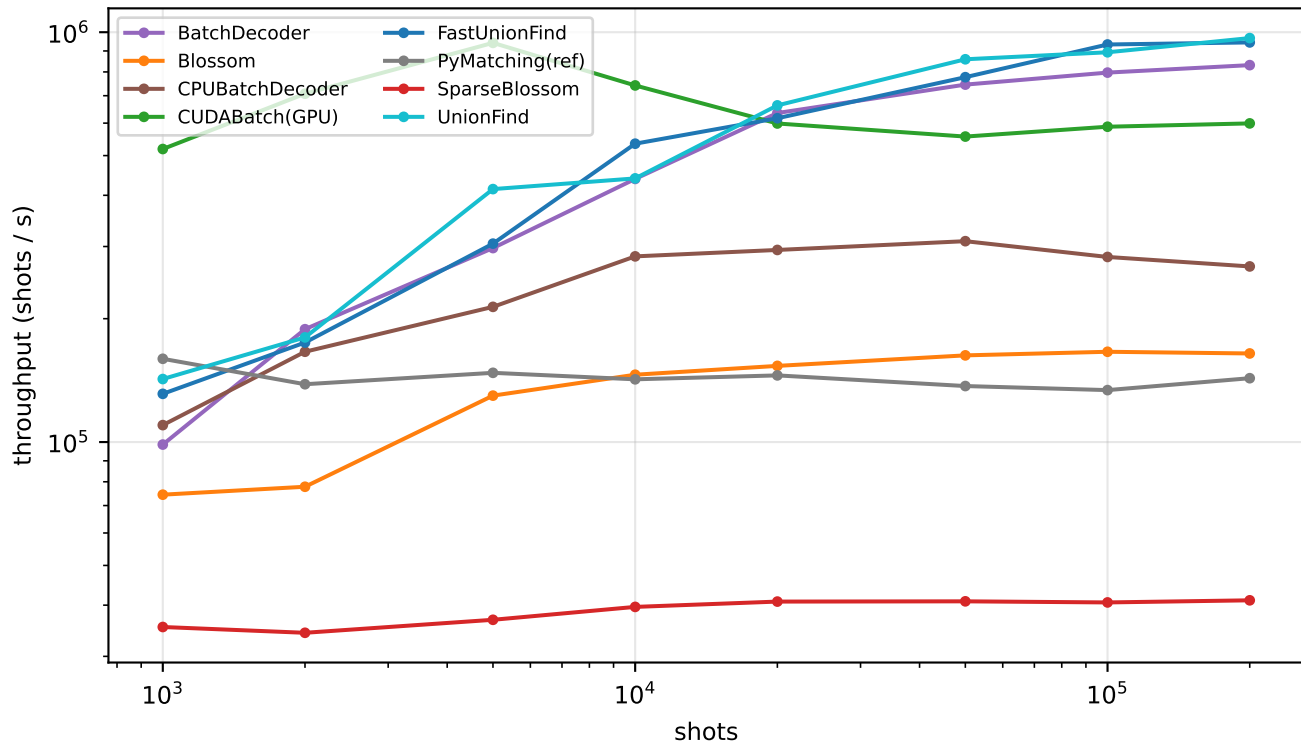
Reproduce:

```
python -m venv venv && venv\Scripts\pip install --no-cache-dir \
    "qector-decoder-v3[bench,stim]==0.6.7"
set QECTOR_LICENSE=academic
python bench/bench_main.py          # throughput + faithfulness + bit-exact
python bench/bench_circuit.py      # circuit-level LER (documented dem API)
python bench/make_report.py         # figures + PDF + markdown
```

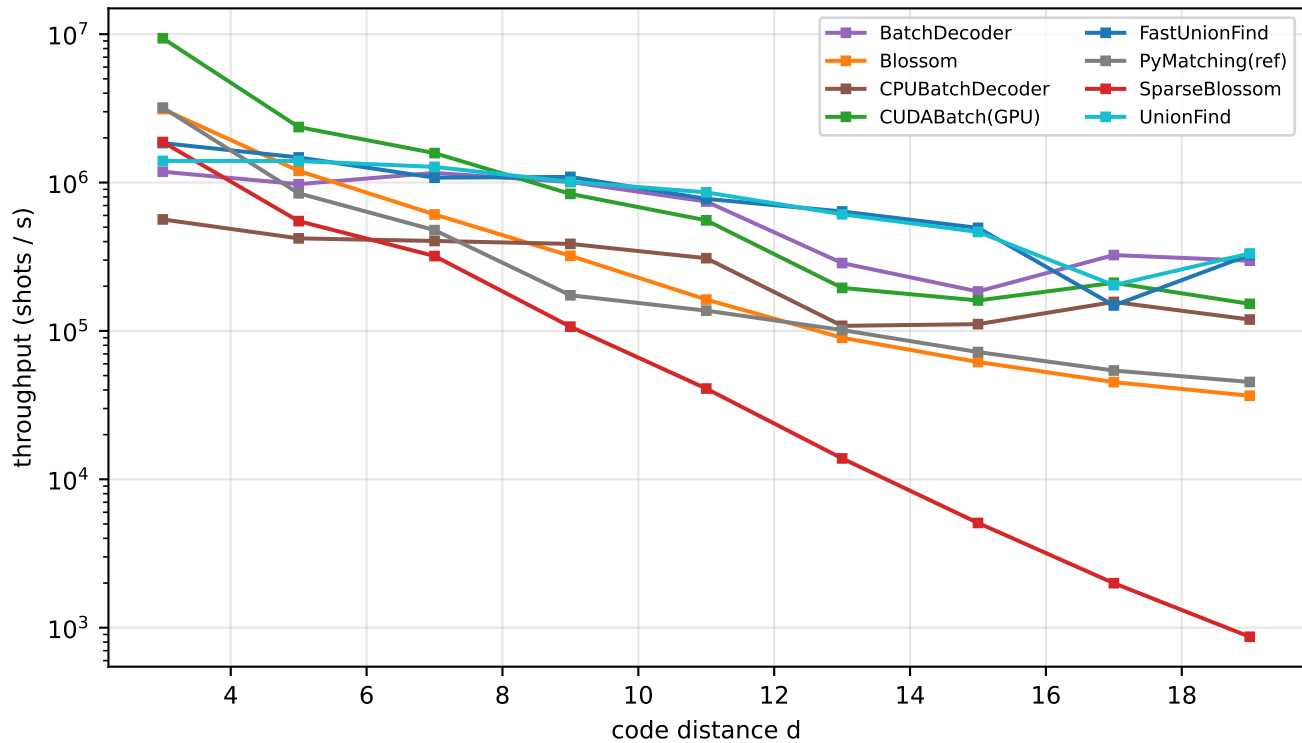
Decoder throughput vs batch size — code-capacity surface d=11



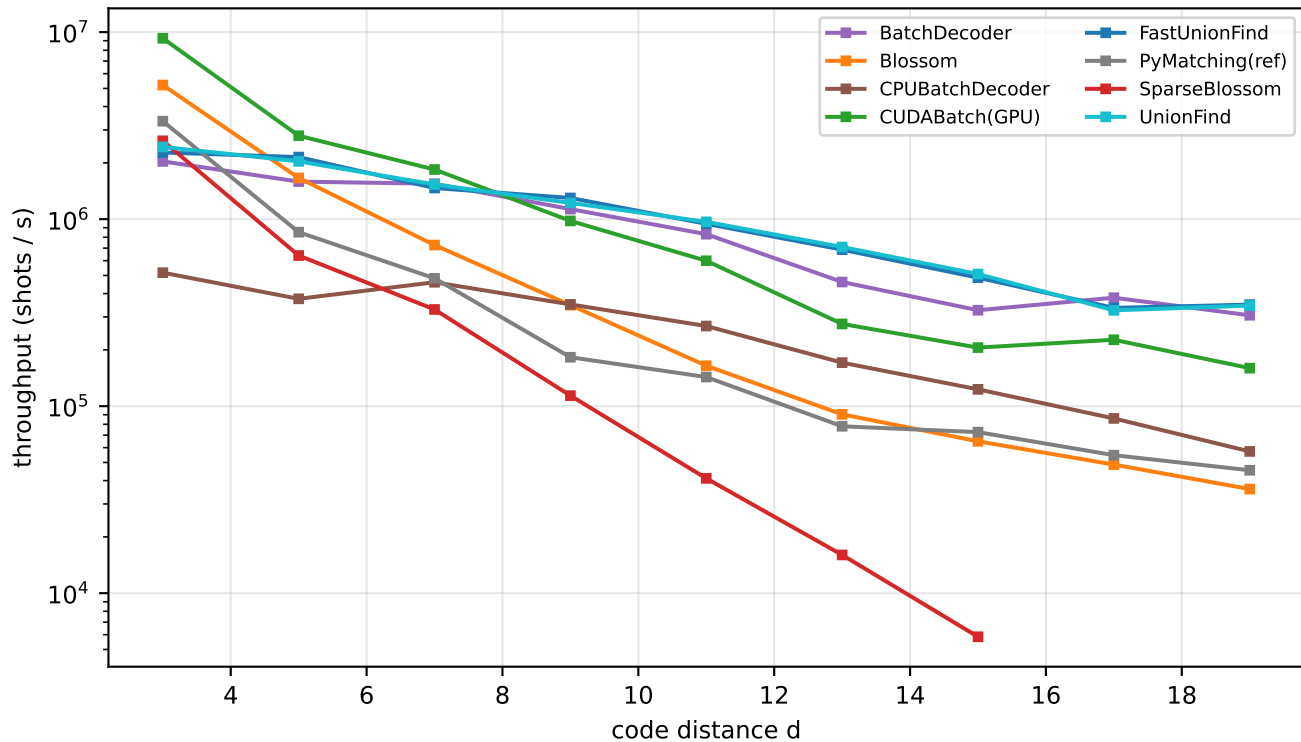
Decoder throughput vs batch size — code-capacity surface d=11



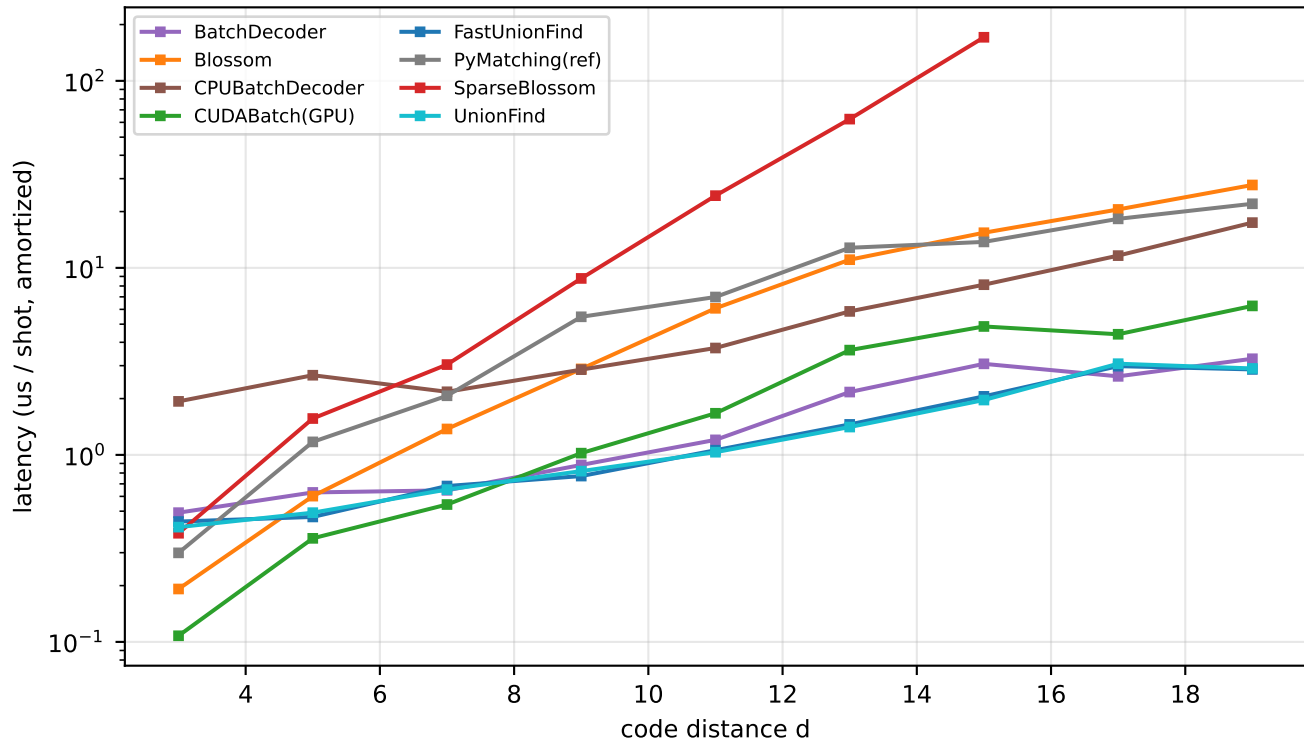
Decoder throughput vs distance — 50,000 shots (code-capacity)



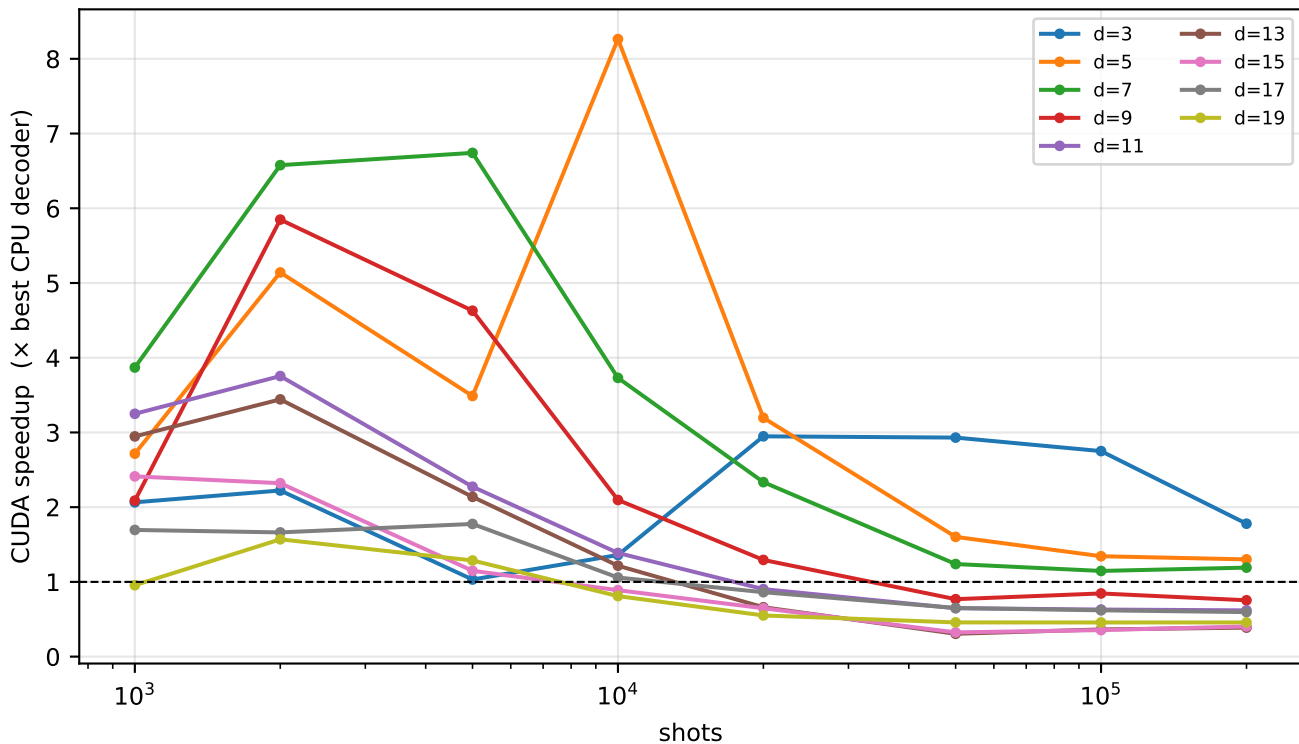
Decoder throughput vs distance — 200,000 shots (code-capacity)



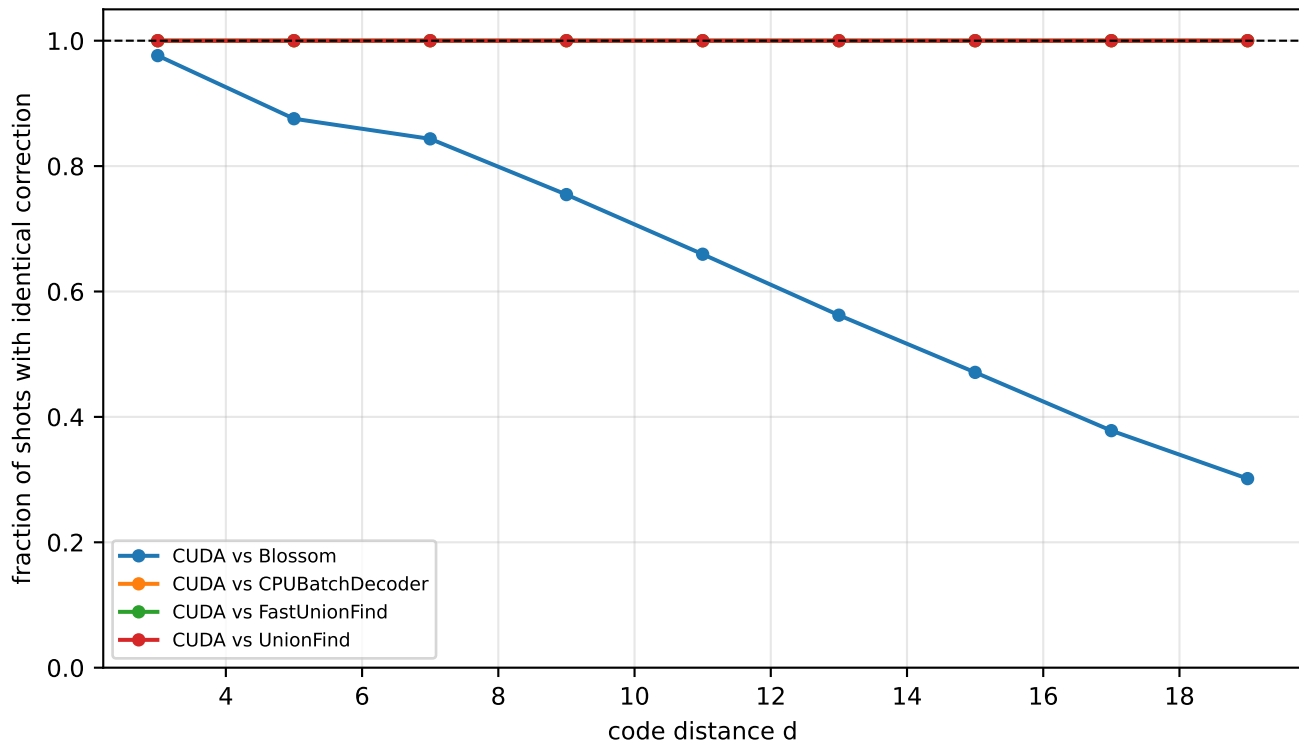
Amortized per-shot latency vs distance — 200,000 shots



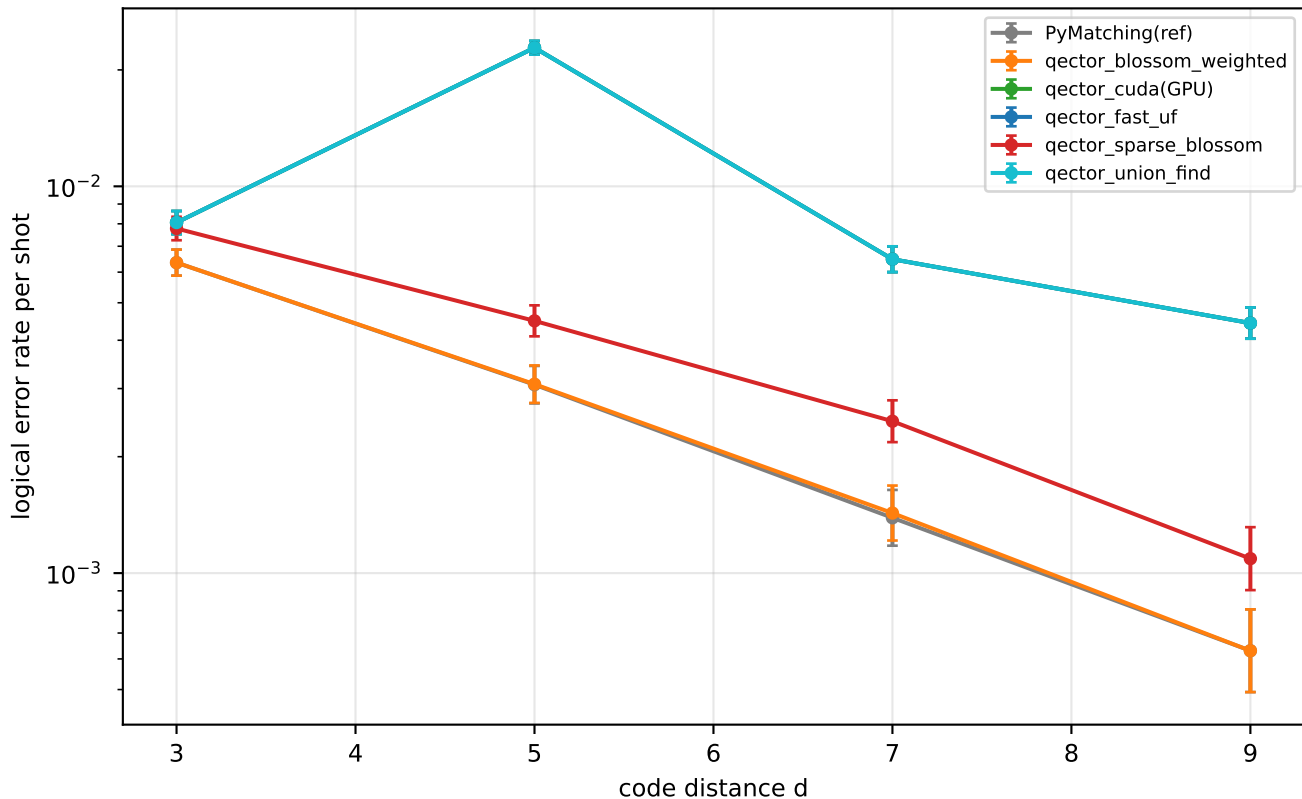
GPU (CUDA) vs best CPU decoder — code-capacity



CUDA vs CPU bit-exactness (row-exact) — 'CUDA bit-identical to CPU' claim



Circuit-level logical error rate vs distance (rotated_memory_z, $p=0.003$)



Throughput table (M shots/s)

Throughput at 200,000 shots (Msh/s), code-capacity

decoder	d3	d5	d7	d9	d11	d13	d15	d17	d19

BPOSD	-	-	-	-	-	-	-	-	-
BatchDecoder	2.04	1.59	1.55	1.13	0.83	0.46	0.33	0.38	0.31
Blossom	5.21	1.66	0.73	0.35	0.16	0.09	0.06	0.05	0.04
CPUBatchDecoder	0.52	0.38	0.46	0.35	0.27	0.17	0.12	0.09	0.06
CUDABatch(GPU)	9.27	2.79	1.84	0.98	0.60	0.28	0.21	0.23	0.16
FastUnionFind	2.27	2.15	1.47	1.30	0.94	0.69	0.49	0.34	0.35
PyMatching(ref)	3.34	0.85	0.48	0.18	0.14	0.08	0.07	0.05	0.05
SparseBlossom	2.62	0.64	0.33	0.11	0.04	0.02	0.01	-	-
UnionFind	2.43	2.04	1.53	1.22	0.97	0.71	0.51	0.33	0.34

Faithfulness & GPU Bit-Exactness

SYNDROME FAITHFULNESS (fraction of shots with $H \cdot c == s$)[illegible]

CUDA vs CPU BIT-EXACTNESS (row-exact fraction of corrections)

[illegible]

Circuit-level LER table

Circuit-level logical error rate (per shot), rotated_memory_z

decoder	d3	d5	d7	d9

PyMatching(ref)	6.35e-03	3.07e-03	1.39e-03	6.30e-04
qector_blossom_weighted	6.35e-03	3.08e-03	1.43e-03	6.30e-04
qector_cuda(GPU)	8.06e-03	2.28e-02	6.48e-03	4.43e-03
qector_fast_uf	8.06e-03	2.28e-02	6.48e-03	4.43e-03
qector_sparse_blossom	7.78e-03	4.49e-03	2.47e-03	1.09e-03
qector_union_find	8.06e-03	2.28e-02	6.48e-03	4.43e-03

shots per point: [(3, 100000)]

95% Wilson intervals in accuracy_circuit.json